

ASSIGNMENT 4: GREEDY ALGORITHMS

Instructor: Orhan Özgüner

Due: November 23 before 11:59 PM

Problem 1

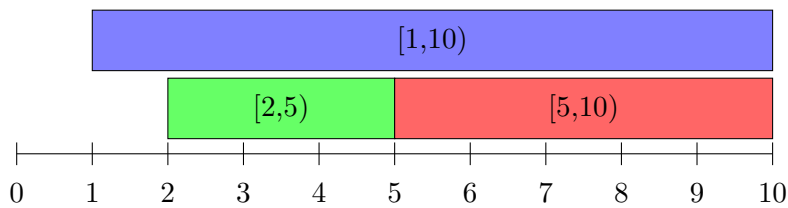
We started the activity selection problem in class. Prove that the following greedy choices do not lead to optimal solutions for the activity selection problem:

- (a) Select the activity with the earliest starting time.
- (b) Select the activity with least duration.
- (c) Select the activity that overlaps the fewest other remaining activities.

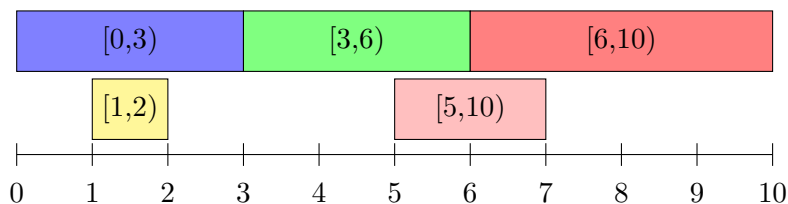
Problem 1 Solution

Reminder: Any proof by counterexample is correct. One possible solution is below:

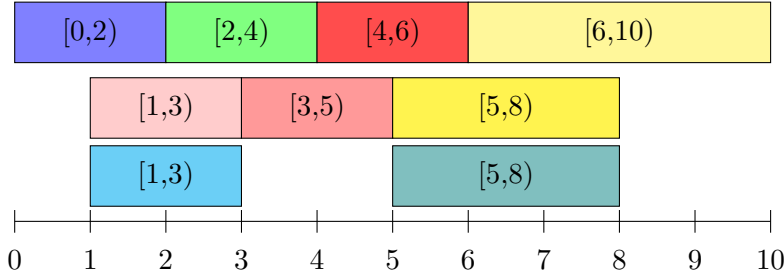
- (a) *Proof* The set of activities on the interval $[0, 10]$ given by $\{[1, 10), [2, 5), [5, 10)\}$ allows a maximum of 2 non-overlapping activities ($\{[2, 5), [5, 10)\}$), but selecting the earliest starting activity selects only the set $\{[1, 10)\}$. This contains 1 activity which is not optimal.



- (b) *Proof* The set of activities on the interval $[0, 10]$ given by $\{[0, 3), [3, 6), [6, 10), [1, 2), [5, 7)\}$ allows a maximum of 3 non-overlapping activities ($\{[0, 3), [3, 6), [6, 10)\}$), but selecting the shortest-duration valid activity, selects only 2 activities: $\{[1, 2), [5, 7)\}$. This is not optimal.



- (c) *Proof* The set of activities on the interval $[0, 10]$ given by $\{[0, 2), [2, 4), [4, 6), [6, 10), [1, 3), [3, 5), [5, 8), [5, 8)\}$ has a maximum of 4 non-overlapping activities: $\{[0, 2), [2, 4), [4, 6), [6, 10)\}$. However, selecting the activities with lowest overlap first selects $[3, 5)$ as it overlaps with only 2 other activities, then can only select a pair of activities of the form (up to reordering) $[1, 3)$ and $[5, 8)$ or $[0, 2)$ and $[6, 10)$ or $[1, 3)$ and $[6, 10)$ or $[0, 2)$ and $[5, 8)$. This leads to selecting only 3 activities, which is not optimal.



Problem 2

You are given two sets A and B , each containing n positive integers. You can choose to reorder each set however you like. After reordering let a_i be the i^{th} element of set A and b_i the i^{th} element of set B . You then receive a payout of

$$\prod_{i=0}^n a_i^{b_i} \quad (1)$$

Give an algorithm that maximizes your profit.

Problem 2 Solution

Greedy Choice Property

Given two sets A and B , match the largest remaining a_i to the largest remaining b_i .

Proof of Greedy Choice Property

Proof. We will designate the largest values in the two arrays, a_x and b_i , respectively. A greedy matching system O will pair these two values together. Now, assume that there exists a non-greedy matching system O' , which has a total profit greater than O (that is, O is not optimal). In order to violate the greedy condition, O' cannot match a_x and b_i to each other, so it must match them with some other values a_y and b_j . So, we have the following setup:

$$O : b_i \iff a_y \quad / \quad b_j \iff a_x$$

$$O' : b_i \iff a_x \quad / \quad b_j \iff a_y$$

where, since a_x and b_i are the largest elements in their respective sets, it is the case that $b_i \geq b_j$ and $a_x \geq a_y$.

Then, since we are assuming that O' has a total profit greater than O , we must also have:

$$a_y^{b_i} \cdot a_x^{b_j} > a_x^{b_i} \cdot a_y^{b_j}$$

Dividing both sides:

$$\frac{a_y^{b_i} \cdot a_x^{b_j}}{a_x^{b_i} \cdot a_y^{b_j}} > 1$$

Now, simplifying:

$$\frac{a_y^{b_i} \cdot a_x^{b_j}}{a_x^{b_i} \cdot a_y^{b_j}} = \frac{a_y^{b_i} \cdot a_x^{b_j}}{a_y^{b_j} \cdot a_x^{b_i}} = a_y^{b_i-b_j} \cdot a_x^{b_j-b_i} = a_y^{b_i-b_j} \cdot a_x^{-(b_i-b_j)} = \left(\frac{a_y}{a_x}\right)^{b_i-b_j} > 1$$

But, at the beginning, we stated that $a_x \geq a_y$, which means that the fraction $\frac{a_y}{a_x}$ is less than or equal to one. Also, since $b_i \geq b_j$, the term $b_i - b_j$ is zero or positive. This implies that:

$$\left(\frac{a_y}{a_x}\right)^{b_i-b_j} \leq 1$$

because a fraction raised to a non-negative exponent can never be greater than one.

This is a contradiction. So, no solution non-greedy solution O' , that is better than the greedy solution O , can exist. This confirms that the greedy property holds.

Pseudocode

The greedy choice “given two sets A and B , match the largest remaining a_i to the largest remaining b_i ”, is equivalent to simply sorting both A and B (both will have the largest elements in position 1, the second-largest in position 2, etc.). Any efficient, in-place sorting algorithm can be used for this purpose, such as in-place merge sort. If we want to return the resulting value as well, we can then just perform element-wise exponentiation of the sorted arrays in a for-loop:

Algorithm 1 Find maximal cross-exponentiation of two sets A and B

```

1: procedure MAXEXPONENT( $A, B$ ):
2:   INPLACEMERGESORT( $A$ )
3:   INPLACEMERGESORT( $B$ )
4:    $s \leftarrow 0$ 
5:   for  $i \leftarrow 1$  to LENGTH( $A$ ) do
6:      $s \leftarrow a[i]^b[i]$ 
7:   end for
8:   return  $A, B, s$ 
9: end procedure

```

Runtime

We first sort the sets A and B , then make a pass over the sets once, taking the power of each of the n pairs of numbers in constant time. Thus the runtime is dominated by sorting: $\mathbf{O}(n + n \log n) = \mathbf{O}(n \log n)$

Problem 3

We have n activities. Each activity i requires t_i time to complete and has deadline d_i . We would like to schedule the activities, assigning a starting time s_i each. The goal is to minimize the maximum delay in completing any activity; that is, we would like to assign the s_i so that $\max_{1 \leq i \leq n}(\Delta_i)$ is minimized, where $\Delta_i = f_i - d_i$ is the delay for activity i and $f_i = s_i + t_i$ is the finishing time for activity i . Note that we can only perform one activity at a given time (if activity i starts at time s_i , the next scheduled activity has to start at time f_i).

For example, if $t = [10, 5, 6, 2]$ and $d = [11, 6, 12, 20]$, then the optimal solution is to schedule the activities in the order $[2, 1, 3, 4]$ to obtain starting/finishing times $s_i/f_i = [5/15, 0/5, 15/21, 21/23]$ and achieve a maximum delay of 9 (for the third activity).

Give an algorithm that minimizes the maximum delay.

Problem 3 Solution

Greedy Choice Property

We will want to select the activity with the *earliest deadline*, first.

Proof of Greedy Choice Property

In order to prove that the application of the greedy strategy “pick the remaining activity with the earliest deadline” will always yield an optimal solution to this problem, we do *not* need to prove that every possible optimal solution can be generated using the greedy strategy. Instead, we will pick an arbitrary optimal solution S that violates the greedy strategy at some point, and construct a solution S' which fully adheres to the greedy strategy, and is also optimal.

Proof. Let S be an optimal schedule of activities, that does not adhere to the greedy strategy. We will call the activity with the earliest deadline i . To violate the greedy strategy, S will *not* schedule it as the first activity.

We can assume that S does not have any idle time (i.e. all activities occur one-after-the other), because this is a property of all optimal solutions to the activity scheduling problem. Since no activity requires idle time, any solution that *includes* idle time can be made to take *less* time, by removing the idle time and scheduling all the activities, in the same order, consecutively.

Let S' be the schedule that is obtained by moving the i th activity to the front and shifting all activities scheduled before i in S . We will prove that S' is an optimal schedule.

We can divide the activities into three groups:

1. i is the activity with the earliest deadline
2. J is the set of activities that were scheduled before i in S and are shifted back by t_i in S'
3. K is the set of activities that were scheduled after i in S

Let the maximum delay in S be δ and the maximum delay in S' be δ' . We have:

$$\delta = \max(\Delta_i, \max_{j \in J}(\Delta_j), \max_{k \in K}(\Delta_k))$$

Where Δ_m is the delay in activity m within schedule S , and

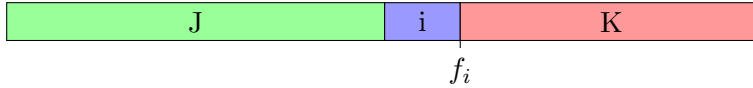
$$\delta' = \max(\Delta'_i, \max_{j \in J}(\Delta'_j), \max_{k \in K}(\Delta'_k))$$

Where Δ'_m is the delay in activity m within schedule S' .

We make the following observations:

1. $\Delta'_i \leq \Delta_i$ since i is scheduled earlier in S' than in S .
2. $\max_{k \in K}(\Delta'_k) = \max_{k \in K}(\Delta_k)$ since $\Delta'_k = \Delta_k$ for all $k \in K$.
3. $\max_{j \in J}(\Delta'_j) \leq \Delta_i$.

Visualization of S :



Visualization of S' :



To visualize the proof for the third inequality, note that J does not “spill over” f_i .

To rigorously prove that the third inequality holds, consider any $j \in J$. We must have $f'_j \leq f_i$, since the set of activities that were scheduled to finish by time f_i in S are the same set of activities in S' (so they all must finish by time f_i).

So we have, for any $j \in J$:

$$\Delta'_j = f'_j - d_j \leq f_i - d_j \leq f_i - d_i = \Delta_i$$

where the last inequality follows from the fact that $d_j \geq d_i$ for any j , as i is the activity with the earliest deadline. So we conclude that

$$\max_{j \in J}(\Delta'_j) \leq \Delta_i.$$

Combining the three observations enumerated above, we have:

$$\delta' = \max(\Delta'_i, \max_{j \in J}(\Delta'_j), \max_{k \in K}(\Delta'_k)) \leq \max(\Delta_i, \Delta_i, \max_{k \in K}(\Delta'_k)) \leq \delta$$

where the last inequality follows from the definition of δ .

Thus, if S is an optimal schedule, S' must also be optimal. We therefore conclude that there is at least one optimal solution, that schedules the activity with the earliest deadline first. $\square$

Pseudocode

Using this greedy choice property, we can sort the activities in non-decreasing order of their deadlines (using any efficient in-place sort, such as an in-place mergesort) and schedule the activities in that order:

Algorithm 2 For lists t (time required) and d (deadlines)

```
1: procedure SCHEDULING( $t, d$ ):  
2:   Create a list of tuples activities from  $t$  and  $d$   
3:   Sort activities by deadline, in ascending order  
4:   Initialize  $currentTime = 0$   
5:   Initialize  $schedule = []$  ▷ To store the order of activities  
6:   for each tuple ( $time, deadline$ ) in activities do  
7:      $start = currentTime$   
8:      $finish = start + time$   
9:      $currentTime = finish$   
10:    Append ( $start, finish$ ) to schedule  
11:   end for  
12:   return schedule  
13: end procedure
```

Runtime

We first sort the activities and then make a pass over the array of activities once, scheduling each activity in constant time. Thus the runtime is dominated by the sorting step, and is

$$\mathbf{O}(n \log n)$$